Complete finite programs for the Janko geometries

Human Compression Project

These are the two full, unchanged mathematical programs required by the companion application. The first establishes the constructions; the second proves the census using the complete retained TomLib subgroup data. Both are charged once in Topic C. The separate execution runner and environment bindings are inventoried separately; they are not hidden in an unpriced claim of complete formal proof.

1 Construction and incidence tests

```
# Entire finite certificate for Compress_C. No geometry census is read.
LoadPackage("tomlib");;
SizeScreen([100000,24]);;
CheckC:=function(test,msg) if not test then Error(msg);fi;end;;
G:=UnderlyingGroup(TableOfMarks("J1"));;
CheckC(Size(G)=175560,"ambient J1 order");;
FindSeedC:=function(K)
 local a,b,c,d,iv,bs;
 a:=GeneratorsOfGroup(K)[1];
 CheckC(Order(a)=2,"ambient first generator");
 iv:=Filtered(Elements(Centralizer(K,a)),x->Order(x)=2);
 for d in iv do
  if d=a then continue;fi;
  bs:=Filtered(Elements(Centralizer(K,d)),x->Order(x)=2 and Order(a*x)=5);
  for c in iv do
   if Order(c*d)<>5 then continue;fi;
   for b in bs do
    if Order(b*c)=3 and (a*b*c)^5=One(K)
      and ((d*c*b)^5*a)^3=One(K) and Group(a,b,c,d)=K then
     return [a,b,c,d];
    fi;
   od;
  od;
 od;
 Error("no source seed");
end;;
s:=FindSeedC(G);;
pairs:=Combinations([1..4],2);;
CheckC(List(pairs,p->Order(s[p[1]]*s[p[2]]))=[5,2,2,3,2,5],"seed pairs");;
CheckC(Normalizer(G,Group(s{[1,2,3]}))=Group(s{[1,2,3]}),"seed facet normalizer");;
# Exact upper bound for the literal source presentation.
F:=FreeGroup("a","b","c","d");; u:=GeneratorsOfGroup(F);;
rels:=Concatenation(List(u,x->x^2),List([1..6],i->
 (u[pairs[i][1]]*u[pairs[i][2]])^[5,2,2,3,2,5][i]),
 [(u[1]*u[2]*u[3])^5,((u[4]*u[3]*u[2])^5*u[1])^3]);;
Q:=F/rels;; q:=GeneratorsOfGroup(Q);;
ct:=CosetTable(Q,Subgroup(Q,q{[2,3,4]}));;
CheckC(Length(ct[1])=1463,"presentation coset index");;
CheckC(Size(Group(List([1,3,5,7],i->PermList(ct[i]))))=175560,"presentation image");;
w:=(s[2]*s[3]*s[4])^5;;
CheckC(Order(w)=2 and ForAll(s{[2,3,4]},x->Comm(w,x)=()),"twist center");;
marks:=[s,[s[1],w*s[2],s[3],s[4]],
 [s[1],s[2],s[3],s[3]^s[4]],
 [s[2]^s[1],s[2],s[3],s[4]],
 [s[2]^s[1],s[2],s[3],s[3]^s[4]],
 [(w*s[2])^s[1],w*s[2],s[3],s[4]]];;
CheckC(Intersection(Group(s{[2,3,4]}),Group(s{[2,3,4]})^s[1])
 =Group(s{[3,4]}),"left overlap");;
CheckC(Intersection(Group(s{[1,2,3]}),Group(s{[1,2,3]})^s[4])
 =Group(s{[1,2]}),"right overlap");;
VerifyC:=function(t)
```

```
 local ss,subs,a,b,ip,pars,i,j,J,rows,T,common,expected,h,pan;
 ss:=Combinations([1..4]);
 subs:=List(ss,J->Group(Concatenation([()],t{J})));
 ip:=0;
 for a in [1..16] do for b in [1..16] do
  if Intersection(subs[a],subs[b])<>subs[Position(ss,Intersection(ss[a],ss[b]))]
   then ip:=ip+1;fi;
 od;od;
 pars:=List([1..4],i->Group(t{Difference([1..4],[i])}));
 h:=[];
 for i in [1..4] do
  rows:=[];
  for j in Difference([1..4],[i]) do
   T:=Intersection(pars[i],pars[j]);
   rows[j]:=Set(List(RightTransversal(pars[j],T),x->RightCoset(pars[i],x)));
  od;
  for J in Combinations(Difference([1..4],[i])) do
   if Length(J)<2 then continue;fi;
   common:=Length(Intersection(rows{J})); T:=Intersection(pars{J});
   expected:=Index(T,Intersection(T,pars[i]));
   Add(h,[i,J,common,expected]);
  od;
 od;
 pan:=List([1..4],i->Size(Intersection(pars{Difference([1..4],[i])})));
 return rec(order:=Size(Group(t)),pairs:=List(pairs,p->Order(t[p[1]]*t[p[2]])),
  parabolics:=List(pars,Size),types:=List(pars,StructureDescription),
  ip_failures:=ip,helly:=h,borel:=Size(Intersection(pars)),panels:=pan);
end;;
expectedP:=[[120,20,20,60],[120,20,20,660],[120,20,60,60],
 [120,120,20,60],[120,120,60,60],[120,120,20,660]];;
expectedPairs:=[[5,2,2,3,2,5],[5,2,2,6,2,5],[5,2,2,3,3,5],
 [5,3,2,3,2,5],[5,3,3,3,3,5],[5,6,2,6,2,5]];;
for i in [1..6] do
 CheckC(ForAll(marks[i],x->Order(x)=2),"marker orders");
 r:=VerifyC(marks[i]);
 CheckC(r.order=175560 and r.ip_failures=0,"generation and IP");
 CheckC(r.parabolics=expectedP[i] and r.pairs=expectedPairs[i],"source tables");
 CheckC(r.types=List(r.parabolics,o->["C2 x A5","D20","A5","PSL(2,11)"]
  [Position([120,20,60,660],o)]),"parabolic isomorphism types");
 CheckC(r.borel=1 and r.panels=[2,2,2,2],"thinness");
 CheckC(ForAll(r.helly,z->z[3]=z[4]),"typed Helly");
 Print("GEOMETRY ",i," PAIRS ",r.pairs," PARABOLICS ",r.parabolics,
  " TYPES ",r.types," IP_MEETS 256 HELLY_ROWS ",Length(r.helly)," PASS\n");
od;
CheckC(Length(Set(List(expectedP,SortedList)))=6,"pairwise nonisomorphism");;
bad:=[[s[1],w*s[2],s[3],s[3]^s[4]],[s[1],s[2],s[3],w*s[4]]];;
for i in [1..2] do
 r:=VerifyC(bad[i]);
 CheckC(r.order=175560 and r.ip_failures=8 and r.borel=1,"negative scope");
 CheckC(r.panels=[[2,4,2,2],[2,2,2,4]][i],"negative panel");
 CheckC(ForAll(r.helly,z->z[3]=z[4]),"negative Helly");
 Print("NEGATIVE ",i," IP_FAILURES ",r.ip_failures," PANELS ",r.panels," PASS\n");
od;
Print("COMPRESS_C_J1_PASS\n");
QUIT;
```

2 Exhaustive extension of proper residues

```
# Complete finite J1 geometry census by proper-residue extension.
# Relative to the complete TomLib J1 subgroup table, not a geometry census.
LoadPackage("tomlib");;
SizeScreen([100000,24]);;
tom:=TableOfMarks("J1");;G:=UnderlyingGroup(tom);;
ids:=MaximalSubgroupsTom(tom)[1];;
ns:=Length(OrdersTom(tom));; incidence:=SubsTom(tom);;
derived:=Filtered([1..ns-1],i->not ForAny([i+1..ns-1],j->i in incidence[j]));;
if derived<>SortedList(ids) or ns<>40 then Error("maximal subgroup catalogue");fi;
Print("MAXIMAL_CLASSES ",derived," ORDERS
 ↪  ",List(ids,i->Size(RepresentativeTom(tom,i))),"\n");;
CheckCensus:=function(t)
 local n,I,i,j,k,K,L,ss,subs,P,rows,J,common,T;
 n:=Length(t); I:=[1..n];
```

```
 # Irredundance is necessary for IP and rejects most global extensions.
 for i in I do
  K:=Group(t{Difference(I,[i])});
  if t[i] in K then return fail;fi;
 od;
 ss:=Combinations(I);
 subs:=List(ss,J->Group(Concatenation([()],t{J})));
 for i in [1..Length(ss)] do for j in [i+1..Length(ss)] do
  if Intersection(subs[i],subs[j])<>subs[Position(ss,Intersection(ss[i],ss[j]))]
   then return fail;fi;
 od;od;
 P:=List(I,i->Group(t{Difference(I,[i])}));
 for i in I do
  rows:=[];
  for j in Difference(I,[i]) do
   K:=Intersection(P[i],P[j]);
   rows[j]:=Set(List(RightTransversal(P[j],K),x->RightCoset(P[i],x)));
  od;
  for J in Combinations(Difference(I,[i])) do
   if Length(J)<2 then continue;fi;
   T:=Intersection(P{J});
   if Length(Intersection(rows{J}))<>Index(T,Intersection(T,P[i]))
    then return fail;fi;
  od;
 od;
 return rec(marking:=t,order:=Size(Group(t)),parabolics:=List(P,Size));
end;;
localReps:=rec();;
for k in [3,4] do
 localReps.(String(k)):=[];
 for id in ids do
  M:=RepresentativeTom(tom,id);
  iv:=Filtered(Elements(M),x->Order(x)=2);
  A:=Image(ActionHomomorphism(M,iv,OnPoints));
  oo:=OrbitsDomain(A,Combinations([1..Length(iv)],k),OnSets);
  count:=0;
  for o in oo do
   t:=iv{o[1]}; r:=CheckCensus(t);
   if r<>fail then Add(localReps.(String(k)),t);count:=count+1;fi;
  od;
  Print("LOCAL ",k," MAXIMAL ",id," ORDER ",Size(M)," ORBITS ",Length(oo)," REGULAR
   ↪  ",count,"\n");
 od;
 Print("LOCAL_TOTAL ",k," ",Length(localReps.(String(k))),"\n");
od;
Print("LOCAL_ENUMERATION_DONE\n");
involClasses:=Filtered(ConjugacyClasses(G),c->Order(Representative(c))=2);;
if Length(involClasses)<>1 then Error("single involution class required");fi;
ivG:=SortedList(Elements(involClasses[1]));;
if Length(ivG)<>1463 then Error("involution count");fi;
a0:=ivG[1];; C0:=Elements(Centralizer(G,a0));;
if Length(C0)<>120 then Error("involution centralizer");fi;
conjCache:=[];;
CanonicalCensus:=function(t)
 local ans,x,p,g,c,key;
 ans:=fail;
 for x in t do
  p:=PositionSorted(ivG,x);
  if not IsBound(conjCache[p]) then
   g:=RepresentativeAction(G,x,a0);
   if g=fail or x^g<>a0 then Error("involution conjugator");fi;
   conjCache[p]:=g;
  fi;
  g:=conjCache[p];
  for c in C0 do
   key:=SortedList(List(t,y->PositionSorted(ivG,y^(g*c))));
   if ans=fail or key<ans then ans:=key;fi;
  od;
 od;
 return ans;
end;;
globalClasses:=[];;fiveClasses:=[];;classParabolics:=[];;
for k in [3,4] do
 keys:=[];unique:=[];
```

```
 for t in localReps.(String(k)) do
  key:=CanonicalCensus(t);
  if not key in keys then Add(keys,key);Add(unique,t);fi;
 od;
 Print("GLOBAL_LOCAL_ORBITS ",k," ",Length(unique),"\n");
 tested:=0;passed:=0;start:=Runtime();
 for i in [1..Length(unique)] do
  t:=unique[i];K:=Group(t);
  for x in ivG do
   if x in K then continue;fi;
   tested:=tested+1;
   r:=CheckCensus(Concatenation(t,[x]));
   if r=fail then continue;fi;
   if k=3 and r.order<>175560 then continue;fi;
   passed:=passed+1;
   key:=CanonicalCensus(r.marking);
   if k=3 and not key in globalClasses then
    Add(globalClasses,key);Add(classParabolics,SortedList(r.parabolics));
    Print("NEW_RANK4 ",key," PARABOLICS ",SortedList(r.parabolics),"\n");
   elif k=4 and not key in fiveClasses then
    Add(fiveClasses,key);
    Print("NEW_RANK5 ",key," GROUP_ORDER ",r.order," PARABOLICS ",r.parabolics,"\n");
   fi;
  od;
  Print("EXTENSION ",k+1," LOCAL ",i,"/",Length(unique)," TESTED ",tested,
   " PASSED ",passed," SECONDS ",(Runtime()-start)/1000,"\n");
 od;
 Print("STAGE_TOTAL ",k+1," TESTED ",tested," ACCEPTED ",passed,"\n");
od;
Print("RANK4_CLASSES ",Length(globalClasses)," RANK5_CLASSES ",Length(fiveClasses),"\n");
if Length(globalClasses)<>6 or Length(fiveClasses)<>0 then Error("geometry
 ↪  classification");fi;
expected:=[[20,20,60,120],[20,20,120,660],[20,60,60,120],
 [20,60,120,120],[60,60,120,120],[20,120,120,660]];;
if SortedList(classParabolics)<>SortedList(expected) then Error("six distinct types");fi;
PrintTo("phase40_compress_operations_exhaustive_classes.json",
 [globalClasses,fiveClasses,classParabolics],"\n");;
Print("COMPRESS_C_EXHAUSTIVE_PASS\n");
QUIT;
```